N-Puzzle & A* Search

Practice Problem Set with Solutions

CSE 318 · Offline 1 Follow-up
14 Problems · Easy to Hard · 30–40 min format
Based on: board.py, search.py, heuristics.py, main.py

PART I — WARM-UP (OUTPUT & INSTRUMENTATION)

Q1. Node Count Reporting Easy

Modify `main.py` so it prints the number of nodes expanded by the search, immediately after the move count line.

SOLUTION

```
print("Nodes expanded = " + str(nodes_expanded))
```

solve() already returns nodes_expanded — just print it.

Q2. Move Sequence Output Easy

Instead of printing every intermediate board, print only the sequence of moves as direction labels (Up / Down / Left / Right), where the direction refers to which way the blank tile moved.

SOLUTION

```
def moves_from_path(path, k):
    moves = []
    for i in range(1, len(path)):
        diff = path[i].index(0) - path[i-1].index(0)
        if diff == -k: moves.append("Up")
        elif diff == k: moves.append("Down")
        elif diff == -1: moves.append("Left")
        elif diff == 1: moves.append("Right")
    return moves
```

Direction blank moved between consecutive boards. Invert if the spec wants tile-direction instead.

Q3. Max Open List Size Easy

Track and report the maximum size the open list (priority queue) reached during the search, as a proxy for space complexity.

SOLUTION

```
max_open_size = 0
while len(open_list) > 0:
    max_open_size = max(max_open_size, len(open_list))
    ...
return path, g, nodes_expanded, max_open_size
```

Sample heap size each loop iteration inside solve(); update the return signature and main.py's unpacking.

PART II — HEURISTICS

Q4. New Admissible Heuristic Easy

Add `row_col_misplaced` to `heuristics.py`: for each tile, add 1 if it's in the wrong row and 1 if it's in the wrong column. Register it in `HEURISTICS` and prove admissibility.

SOLUTION

```
def row_col_misplaced(board, k, goal_pos=None):
    goal_pos = goal_pos or _goal_positions(k)
    total = 0
    for i, val in enumerate(board):
        if val == 0: continue
        row, col = i // k, i % k
        grow, gcol = goal_pos[val]
        total += (row != grow) + (col != gcol)
    return total

HEURISTICS["row_col_misplaced"] = row_col_misplaced
```

Admissible: each tile contributes 0, 1, or 2, always ≤ its Manhattan distance.

Q5. Heuristic Comparison Table Medium

Run every heuristic in `HEURISTICS` on one board and print name, cost, and nodes expanded.

SOLUTION

```
def compare_all(board, k, weight=1.0):
    print(f"{'Heuristic':<18}{'Cost':<8}{'Nodes'}")
    for name, func in HEURISTICS.items():
        path, cost, nodes = solve(board, k, func, weight)
        print(f"{name:<18}{cost:<8}{nodes}")
```

Q6. Break Manhattan's Admissibility Medium

Give a modification that makes Manhattan distance inadmissible, and explain why A* is no longer optimal.

SOLUTION

Multiply h(n) by a constant c > 1 uncapped. Once h' can overestimate true remaining cost, A* can commit to a longer path early because it looks cheaper — admissibility (h ≤ true cost) is broken, so optimality is lost. Weighted A* is different: it deliberately accepts bounded suboptimality (cost ≤ W × optimal) as a tradeoff, not a claim that h stays admissible.

Q7. Effective Branching Factor Medium

Solve for b in 1 + b + b^2 + ... + b^depth = nodes_expanded using binary search.

SOLUTION

```
def effective_branching_factor(nodes_expanded, depth, tol=1e-3):
    if depth == 0: return 0.0
    lo, hi = 1.0, 10.0
    for _ in range(100):
        b = (lo + hi) / 2
        total = sum(b**d for d in range(1, depth + 1))
        if abs(total - nodes_expanded) < tol: return b
        if total < nodes_expanded: lo = b
        else: hi = b
    return b
```

PART III — SEARCH VARIANTS

Q8. Greedy Best-First Search Medium

Implement `solve_greedy` ordering purely by h(n). Compare against A* and state whether it's still optimal.

SOLUTION

```
def solve_greedy(start, k, h_func):
    goal = goal_state(k)
    counter = itertools.count()
    came_from = {start: None}
    visited = {start}
    open_list = [(h_func(start, k), next(counter), start)]
    nodes_expanded = 0
    while open_list:
        _, _, board = heapq.heappop(open_list)
        nodes_expanded += 1
        if board == goal:
            path = []
            cur = board
            while cur is not None:
                path.append(cur); cur = came_from[cur]
            path.reverse()
            return path, len(path) - 1, nodes_expanded
        for nb in neighbors(board, k):
            if nb not in visited:
                visited.add(nb)
                came_from[nb] = board
                heapq.heappush(open_list, (h_func(nb, k), next(counter),
nb))
    return None, -1, nodes_expanded
```

Ignores g(n) entirely — usually fewer nodes, but not optimal.

Q9. Time-Limited Search Medium

Wire the existing `time_limit` parameter into `main.py` and handle a cutoff gracefully.

SOLUTION

```
path, cost, nodes_expanded = solve(board, k, h_func, weight, time_limit=30)
if path is None:
    print("No solution found within time limit")
    return
```

solve() already checks a deadline on each pop; just pass the argument and handle None.

Q10. IDA* Implementation Hard

Implement Iterative Deepening A* as a memory-efficient alternative to A*.

SOLUTION

```python
def solve_ida(start, k, h_func):
    goal = goal_state(k)
    nodes_expanded = 0

    def search(path, g, bound):
        nonlocal nodes_expanded
        board = path[-1]
        f = g + h_func(board, k)
        if f > bound: return f
        if board == goal: return "FOUND"
        nodes_expanded += 1
        minimum = float('inf')
        for nb in neighbors(board, k):
            if len(path) >= 2 and nb == path[-2]:
                continue
            path.append(nb)
            t = search(path, g + 1, bound)
            if t == "FOUND": return "FOUND"
            minimum = min(minimum, t)
            path.pop()
        return minimum

    bound = h_func(start, k)
    path = [start]
    while True:
        t = search(path, 0, bound)
        if t == "FOUND":
            return path, len(path) - 1, nodes_expanded
        if t == float('inf'):
            return None, -1, nodes_expanded
        bound = t
```

DFS bounded by an f-cost threshold, raised each iteration; O(depth) memory instead of A*'s exponential open/closed sets.

PART IV — BOARD & SOLVABILITY EDGE CASES

Q11. Alternate Goal Convention Hard

Generalize `goal_state(k)` for blank at top-left. Does `is_solvable()`'s parity rule still hold?

SOLUTION

```python
def goal_state(k, blank_at_start=False):
    if blank_at_start:
        return tuple([0] + list(range(1, k * k)))
    return tuple(list(range(1, k * k)) + [0])
```

The parity rule does NOT transfer unmodified — the blank-row term assumes the goal blank is the last cell. Re-derive blank_row_bottom relative to the new goal, or verify with a brute-force BFS check on a small board.

Q12. Random Solvable Board Generator Medium

Write `generate_boards(n, k)` producing n distinct, solvable, random k×k boards.

SOLUTION

```python
def random_solvable_board(k):
    while True:
        vals = list(range(k * k))
        random.shuffle(vals)
        board = tuple(vals)
        if is_solvable(board, k):
            return board

def generate_boards(n, k):
    return [random_solvable_board(k) for _ in range(n)]
```

Rejection sampling — roughly half of random permutations are solvable, so cheap in expectation.

PART V — STRETCH

Q13. Weighted A* Suboptimality Bound Hard

Run one board through weighted A* at W = 1.0, 1.2, 2.0, 5.0 and state the theoretical suboptimality bound.

SOLUTION

```
for w in [1.0, 1.2, 2.0, 5.0]:
    path, cost, nodes = solve(board, k, manhattan, weight=w)
    print(w, cost, nodes)
```

Bound: solution cost ≤ W × optimal cost, since f(n) = g(n) + W·h(n) never underestimates true remaining cost by more than a factor of W.

Q14. Pattern Database Heuristic (discussion) Hard

Describe building a pattern database heuristic for the 8-puzzle and why it dominates Manhattan distance.

SOLUTION

Partition tiles into groups (e.g. {1,2,3,4,blank} and {5,6,7,8}). For each group, precompute via backward BFS from the goal the exact minimum moves to place just that group, treating other tiles as don't-care. Store in a lookup table keyed by that group's tile positions; sum PDB values across disjoint groups at runtime. It dominates Manhattan because it captures joint tile-interaction costs that summing independent per-tile distances misses, while remaining exact/admissible for the subproblem.

End of problem set — 14 problems with solutions, ordered easy to hard within each part.